MediSpark

Distributed Multi-Disease Clinical Risk Prediction Engine

Suman Maitreya M Rajkumar R Harish K GRS Kartheek

Course: Big Data Analytics (23AID302)

CB.AI.U4AID24156 CB.AI.U4AID24118 CB.AI.U4AID24143 CB.AI.U4AID24116

Faculty: *Prof. Sreeja B P*

October 19, 2026

Problem Statement

Problem: In healthcare, disease prediction systems typically handle only one condition at a time. A patient visiting a clinic must go through separate screening tools for heart disease, diabetes, stroke, and lung cancer each with its own data format, its own model, and no shared infrastructure. These tools rarely work on large-scale data or provide instant results.

MediSpark solves this by building a single distributed pipeline that:

1. Predicts risk for **4 diseases + X-ray classification** simultaneously
2. Cleans and engineers features on **500K-row datasets** automatically
3. Scores new patients in **real time** every 10 seconds
4. Serves results through a **web dashboard** with PDF reports

Why Big Data?

V	How MediSpark addresses it
Volume	2 million+ rows (4 × 500K SMOTE-augmented) + 5,606 X-ray images
Variety	CSV, JSON, Parquet inputs; structured tabular + unstructured image data
Velocity	Spark Structured Streaming scores patients every 10 seconds
Veracity	Automated cleaning outlier removal, null handling, impossible-value filtering
Value	Unified multi-disease risk scores with explainable predictions & PDF reports

Datasets

Tabular Datasets all SMOTE-augmented to **500,000 rows** each

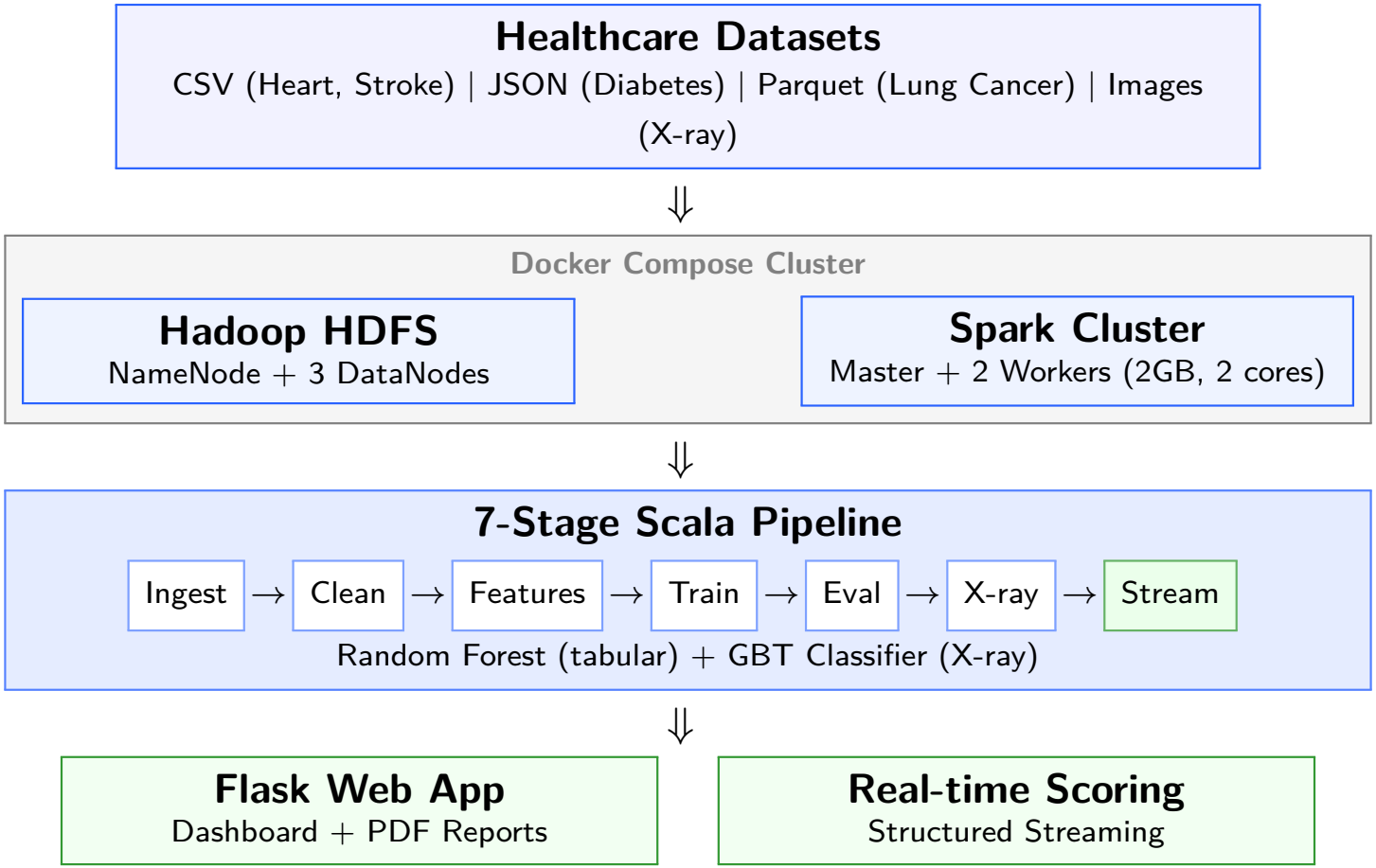
Disease	Format	Columns	Target	Task
Heart Disease	CSV	14	`target` → `label`	Binary (0/1)
Diabetes	JSON	9	`Outcome` → `label`	Binary (0/1)
Stroke	CSV	12	`stroke` → `label`	Binary (0/1)
Lung Cancer	Parquet	26	`Level` → `label`	3-class (Low/Med/High)

X-ray Dataset

- **5,606 images** 2 classes (Normal / Abnormal)
- 16 features per image: 14 handcrafted pixel features (mean, std, skew, kurtosis, entropy, edge density, symmetry, etc.) + 2 demographics (age, gender)
- Stored as **pre-extracted Parquet** on HDFS not raw images

Key attributes: source, size (volume), variety (CSV/JSON/Parquet/image), velocity (streaming input)

System Architecture



Spark Workflow

DataFrame Operations

- All stages use **DataFrames** (not raw RDDs)
- Transforms: `filter`, `withColumn`, `StringIndexer`, `VectorAssembler`
- Actions: `write.parquet`, `model.save`

DAG & Execution

- 84 jobs, 161 completed stages
- Pipeline: `Scan` → `WholeStageCodegen` → `mapPartitions` → `reduceByKey`
- Shuffle write: 259.1 KiB per stage

Caching & Partitioning

- `bagged tree points`: 303.6 MiB cached (MLlib internally caches DataFrame as RDD for tree building)
- Storage: Memory Deserialized 1x Replicated
- 2 partitions across executor
- Constrained: 1 GB driver + 1 GB executor

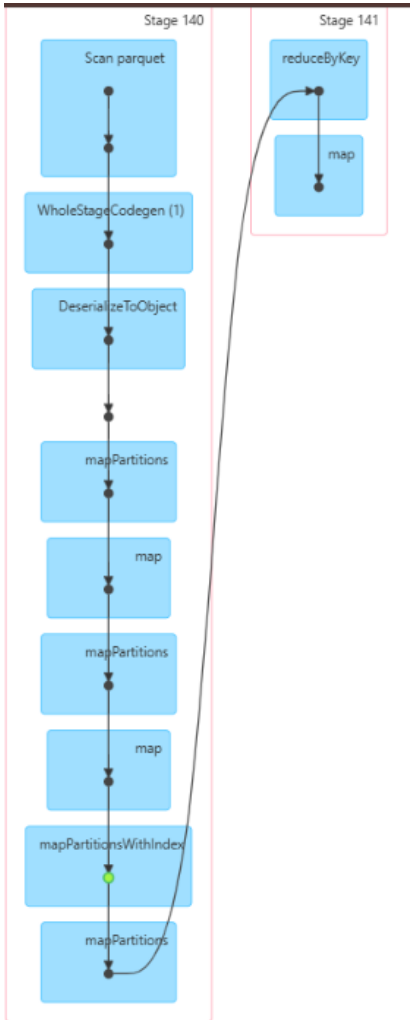


Fig 1: DAG Visualization RandomForest training

Spark Workflow Spark UI Evidence

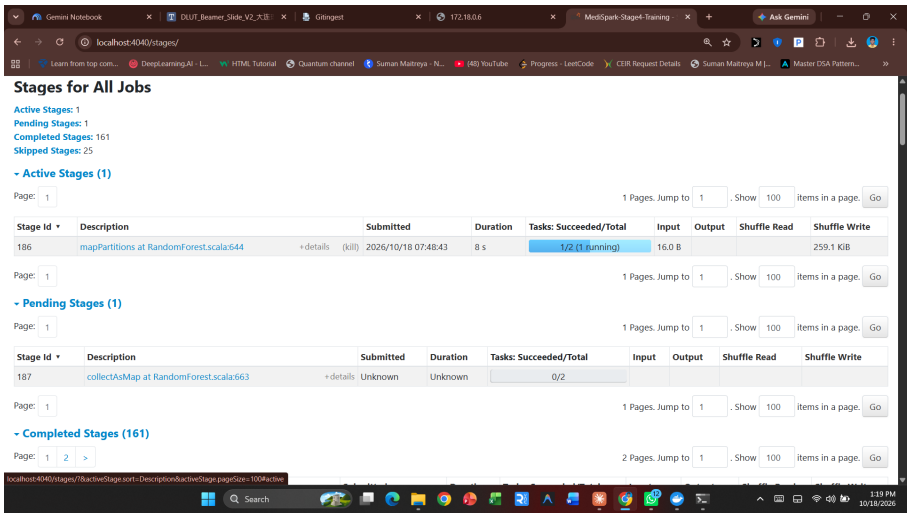


Fig 2: 161 completed stages, shuffle read/write

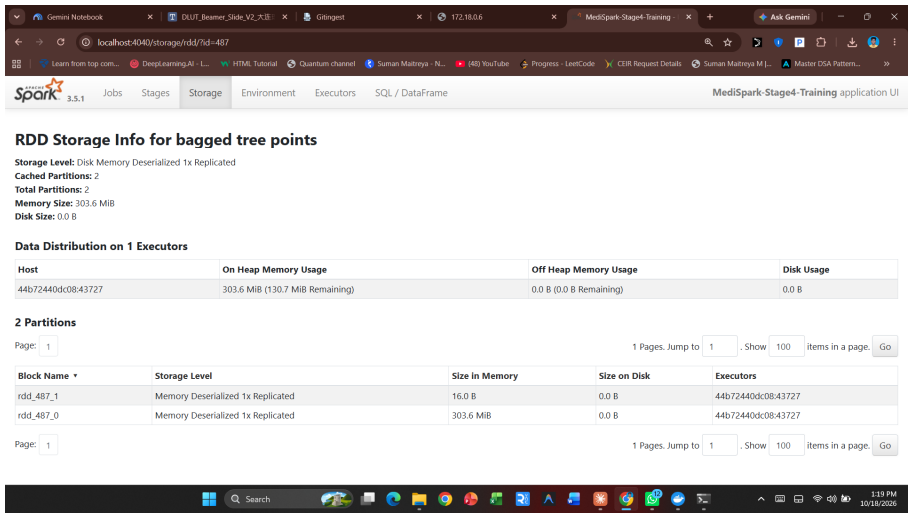


Fig 3: Cached RDD 303.6 MiB, 2 partitions, 100%

Scala Implementation

Data Cleaning (DataCleaning.scala)

```scala
// Fill null BMI with median
val bmiMedian = strokeDf
  .stat.approxQuantile("bmi",
    Array(0.5), 0.01)(0)
strokeDf = strokeDf
  .na.fill(bmiMedian, Seq("bmi"))
strokeDf = strokeDf
  .na.fill("Unknown",
    Seq("smoking_status"))
```

Model Training (ModelTraining.scala)

```scala
val rf = new RandomForestClassifier()
  .setLabelCol("label")
  .setFeaturesCol("features")
  .setNumTrees(numTrees) // 100 or 150
  .setMaxDepth(10)
  .setFeatureSubsetStrategy("sqrt")
val model = rf.fit(train)
model.write.overwrite()
  .save(s"$modelPath/$disease")
```

Feature Engineering

(FeatureEngineering.scala)

```scala
val indexers = catCols.zip(indexedCols)
  .map { case (in, out) =>
  new StringIndexer()
    .setInputCol(in).setOutputCol(out)
    .setHandleInvalid("keep")
}
val assembler = new VectorAssembler()
  .setInputCols(featureCols)
  .setOutputCol("features_raw")
  .setHandleInvalid("skip")
```

Streaming (StreamScoring.scala)

```scala
val query = streamDf.writeStream
  .foreachBatch { (batchDf, batchId) =>
    val heartPreds =
      heartModel.transform(heartAssembled)
    val scored = heartPreds
      .join(diabetesPreds, "patient_id")
      .join(strokePreds, "patient_id")
      .join(lungPreds, "patient_id")
    scored.write.mode("append")
      .parquet(s"$streamOut/batch_$batchId
          ")
  }
  .trigger(ProcessingTime("10 seconds"))
  .start()
query.awaitTermination(120000)
```

ML Pipeline & Results

Algorithms Used

- **Tabular (4 diseases):** Random Forest
  - 100 trees (150 for lung cancer)
  - maxDepth = 10, seed = 42
  - featureSubsetStrategy = sqrt
- **X-ray:** GBT Classifier
  - 200 iterations, maxDepth = 6
  - 16 handcrafted pixel features
  - stepSize = 0.1, subsamplingRate = 0.8

Pipeline Stages

- StringIndexer → VectorAssembler → StandardScaler → RF / GBT
- SMOTE-augmented to 500K rows each
- Leakage-safe train/test split via `_split` tag

Model Performance (Test Set)

Disease	Acc%	F1%	Prec%	AUC%
Heart	99.60	99.60	99.60	99.99
Diabetes	97.45	97.44	97.47	99.70
Stroke	92.13	89.55	92.75	94.57
Lung Cancer	93.57	93.57	93.58	—

Test Set Sizes

Disease	Test Samples
Heart	100,000
Diabetes	95,670
Stroke	99,998
Lung Cancer	100,000

Lung Cancer AUC N/A multi-class (3 classes: Low / Medium / High)

Challenges, Limitations & Future Work

Challenges Faced

- Memory constrained: **1 GB driver + 1 GB executor** required careful caching
- No CNN in Spark MLlib built **16 handcrafted pixel features** + GBT for X-ray
- Stroke class imbalance recall only **17.89%** for positive class

Limitations

- High accuracy may reflect SMOTE-augmented synthetic patterns
- Single-node Docker cluster, not true multi-node deployment

Future Work

- Deploy on **AWS EMR / Azure HDInsight** for real scalability
- Deep learning for X-ray via **ONNX + TorchServe**
- Real-time hospital integration with **Kafka + Spark Streaming**
- Model explainability using **SHAP / LIME**

References

Base Paper

uthor(s) , "[Title of Base Paper]," *[Journal/Conference]*, vol. [X], no. [Y], pp. [XX–YY], [Year]. DOI: [DOI]

Datasets

2. UCI Heart Disease Kaggle (https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data)
3. Pima Indians Diabetes Kaggle (https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database)
4. Stroke Prediction Kaggle (https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset)
5. Lung Cancer Prediction Kaggle (https://www.kaggle.com/datasets/thedevastator/cancer-patients-and-air-pollution)
6. Chest X-ray (Normal/Abnormal) Kaggle (https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia)

Technologies

- Apache Spark 3.5.1, Hadoop 3.2.1, Scala 2.12.18, Spark MLlib
- Docker Compose, Flask, ONNX Runtime

Thank You

Guide: Sreeja B.P | **Course:** 23AID302 Big Data Analytics

GitHub: https://github.com/Suman-Maitreya/MediSpark